

ABSTRACT

To overcome the latency limitations of traditional microcontroller-based architectures, this system leverages hardware-level parallel processing via a priority-based Finite State Machine (FSM) implemented in Verilog HDL. The system continuously monitors critical environmental parameters—including fire, gas leakage, temperature, lighting conditions, and proximity—ensuring immediate, lag-free hazard detection and alert generation.

A key innovation of this work is the integration of a real-time Digital Twin. Through a dedicated UART communication interface, the FPGA transmits real-time system states to a 3D environment modelled in Blender. A custom Python script parses this serial data stream to synchronize virtual components—such as doors, fans, and alarm indicators—with the physical sensor inputs.

This FPGA-to-Blender architecture provides an immersive, photorealistic monitoring interface that bridges the gap between hardware-level logic and virtual visualization. The results demonstrate that FPGA technology offers superior speed, reliability, and modular scalability, establishing an efficient and professional-grade solution for modern smart home automation.

LIST OF FIGURES

Figure	Page No.
Fig. 1.1 System Flowchart -----	6
Fig. 3.1 RealDigital Boolean Board (AMD Spartan-7 FPGA) -----	12
Fig. 3.2 Internal Architecture of AMD Spartan-7 FPGA -----	13
Fig. 3.3 Boolean Board Internal Block Diagram -----	14
Fig. 3.4 HC-SR04 Ultrasonic Sensor Pinout -----	15
Fig. 3.5 IR Flame Sensor Module -----	16
Fig. 3.6 MQ Gas Sensor Module -----	17
Fig. 3.7 KY-028 Temperature Sensor Module -----	18
Fig. 3.8 LDR Light Sensor Module -----	18
Fig. 4.1 Behavioral Simulation -----	26
Fig. 4.2 RTL Analysis -----	26
Fig. 4.3 Synthesis Report -----	27
Fig. 4.4 Implementation Timing Report -----	28
Fig. 4.5 Bitstream Generated Successfully -----	28
Fig. 4.6 Vivado Hardware Manager -----	29
Fig. 4.7 FPGA Programmed Successfully -----	29
Fig. 5.1 Reset Verification (Hardware and Blender Output) -----	33
Fig. 5.2 Fire Detection (Hardware and Blender Output) -----	34
Fig. 5.3 Gas Detection (Hardware and Blender Output) -----	35
Fig. 5.4 Temperature Detection (Hardware and Blender Output) -----	35
Fig. 5.5 Light Detection (Hardware and Blender Output) -----	36
Fig. 5.6 Object Detection (Hardware and Blender Output) -----	37
Fig. 5.7 FPGA Resource Utilization Report -----	38
Fig. 5.8 Timing Summary Report -----	39
Fig. 5.9 UART Results -----	40

Figure	Page No.
Fig. 5.10 Fire Alarm Visual in Blender -----	40
Fig. 5.11 Door Visual in Blender -----	41
Fig. 5.12 Light Visual in Blender -----	41
Fig. 5.13 Temperature Visual in Blender -----	42
Fig. 5.14 Gas Sensor Visual in Blender -----	42

LIST OF TABLES

Table	Page No.
Table 3.1 Specifications of the Boolean Board -----	15
Table 3.2 Specifications of HC-SR04 -----	16
Table 3.3 Specifications of IR Flame Sensor Module -----	16
Table 3.4 Specifications of MQ Gas Sensor -----	17
Table 3.5 Specifications of KY-028 Temperature Sensor -----	18
Table 3.6 Specifications of LDR Sensor Module -----	19
Table 3.7 Functional Test Cases -----	23
Table 3.8 Expected vs Actual Results -----	24

CHAPTER 1

INTRODUCTION

1.1 Background

Modern smart home automation is increasingly vital for improving residential safety, energy efficiency, and overall comfort. To effectively address critical household hazards—such as fire, gas leaks, extreme temperatures, and unauthorized intrusion—systems must detect threats instantaneously to prevent significant damage. This project replaces traditional, latency-prone microcontroller architectures with an AMD Spartan-7 FPGA on a RealDigital Boolean Board, utilizing hardware-level parallel processing to ensure high-speed, deterministic monitoring of multiple environmental sensors. By bridging this robust hardware platform with an immersive Blender-based Digital Twin via a UART serial interface, the system provides an advanced, real-time automation solution that synchronizes physical sensor data with a photorealistic 3D virtual environment for intuitive remote monitoring.

1.2 Problem Statement

Today's home automation systems mostly rely on microcontrollers. While these are popular, they have a major drawback: they process information sequentially, one sensor at a time. If multiple emergencies—like a fire and a gas leak—happen at once, the system's response time slows down. Additionally, many systems on the market only focus on a single hazard instead of offering complete home protection.

There is a clear need for a system that can watch out for multiple environmental and precautions simultaneously. It needs to spot danger instantly, sound the alarm, and be easy for users to monitor. This project solves that problem by using an FPGA, which allows the system to process all sensor data at the exact same time, providing a much faster and more complete safety solution.

1.2.1 Proposed System

We have integrated an IR flame sensor for fire detection, an MQ gas sensor for identifying leaks, a KY-028 sensor to track temperature, an LDR to manage ambient lighting, and an HC-SR04 ultrasonic sensor for proximity and motion detection.

When a sensor detects an anomaly, the FPGA immediately updates the onboard LEDs, activates the RGB indicator, and displays a status code on the seven-segment display. Simultaneously,

the system relays this status to a computer dashboard via UART. Because the FPGA handles all these sensor inputs in parallel, it eliminates the processing delays common in traditional microcontrollers, ensuring a fast, reliable, and highly responsive network.

1.2.2 Digital Twin Concept

A Digital Twin is a high-fidelity virtual representation that mirrors the state, behaviour, and physical configuration of a real-world system in real-time. The Digital Twin serves as an immersive monitoring interface, bridging the gap between the invisible logic inside the FPGA and the user's physical environment. A Python-driven 3D model in Blender updates instantly to reflect the home's current status—such as activating virtual lights, rotating fans, or triggering fire alerts.

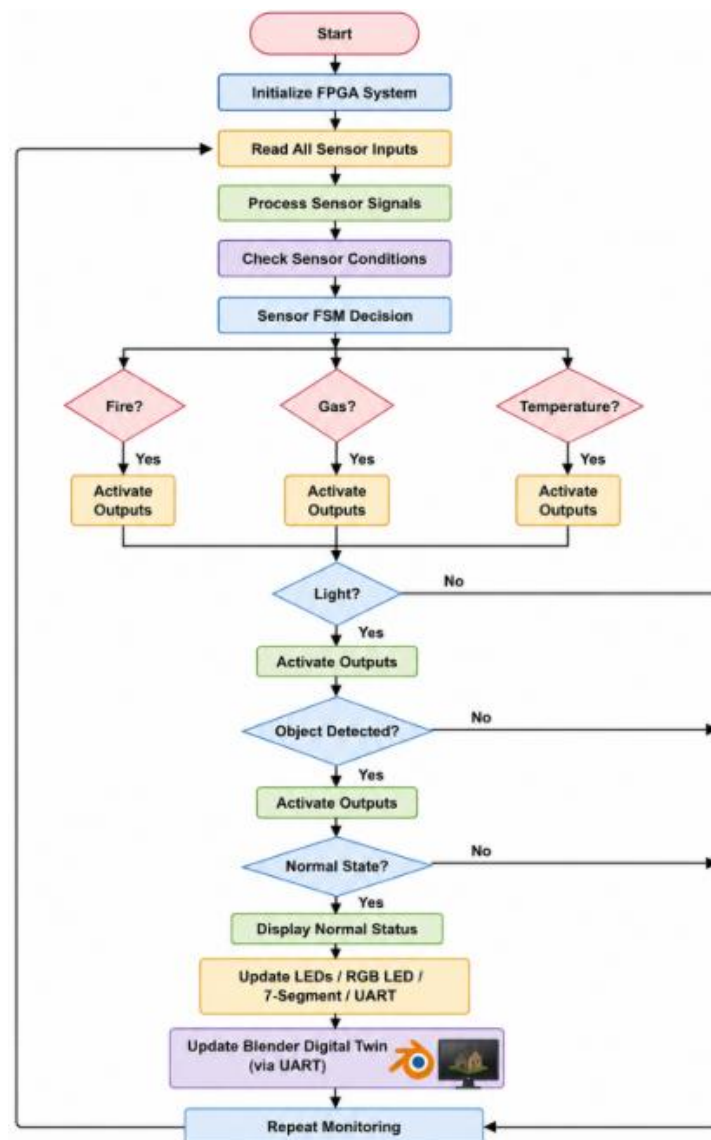


Fig. 1.1: System Flowchart

1.3 Objectives

The main objective of this project is to design and implement a Home Automation and Security System using the AMD Spartan-7 FPGA.

The specific objectives are:

- To detect fire using an IR flame sensor.
- To identify gas leakage using an MQ gas sensor.
- To monitor surrounding temperature using a KY-028 temperature sensor.
- To detect low-light conditions using an LDR sensor.
- To detect nearby objects using an HC-SR04 ultrasonic sensor.
- To process all sensor inputs simultaneously using FPGA hardware.
- To implement a Finite State Machine (FSM) for system control.
- To generate alerts through LEDs, RGB LED, and seven-segment display.
- To transmit the current system status through UART for visualization using a Blender-based digital twin.

1.4 Scope of Work

This project focuses on building an FPGA-based smart home monitoring system that keeps track of various environmental and security factors in real time. We integrated five different sensors with the RealDigital Boolean Board and used Verilog HDL to manage the logic, ensuring the system can process all sensor data simultaneously.

When the system detects anything out of the ordinary, it immediately provides feedback through visual and audible alerts directly on the board. For remote monitoring, it also sends live status updates to a computer using UART communication. While the current version is optimized for local, board-based monitoring, it is designed to be easily expandable, allowing for future upgrades like wireless connectivity, cloud-based data storage, and mobile app integration.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Home automation has rapidly evolved from a luxury to a necessity, driven by the increasing need for residential safety, energy efficiency, and convenience. Traditional systems typically rely on microcontroller-based architectures; however, as the complexity of monitoring multiple environmental and security threats—such as fire, gas leaks, and unauthorized entry—grows, these sequential processing platforms often struggle with response latency and lack of scalability.

This literature review explores the shift toward Field Programmable Gate Array (FPGA) technology as the preferred platform for required applications. By leveraging parallel processing, FPGAs enable the simultaneous monitoring of critical sensor inputs without the bottleneck effects associated with standard microcontrollers. This review synthesizes current research on sensor-integrated FPGA architectures, demonstrating why hardware-level control is superior for environments where detection speed and system reliability are paramount.

2.2 Home Automation Systems

Home automation systems integrate sensors and controllers to automate domestic environments, enhancing safety, energy efficiency, and user comfort. Modern implementations have evolved from simple appliance control (lights and fans) to holistic security networks capable of detecting fire, gas leaks, and unauthorized entry. By automating responses based on sensor data, these systems reduce the necessity for human intervention and optimize power consumption.

2.3 FPGA-Based Home Automation Systems

Field Programmable Gate Arrays (FPGAs) have emerged as the preferred platform for high-performance home automation due to their parallel processing architecture. Unlike microcontrollers that process instructions sequentially, FPGAs execute logic tasks simultaneously via hardware-level parallelism. Implementing systems using Verilog HDL and Finite State Machines (FSMs) provides the modularity, speed, and deterministic reliability essential for real-time safety-critical applications.

2.4 Digital Twin Technology in Industry 4.0

In the era of Industry 4.0, Digital Twin technology has become a vital component of intelligent systems. A Digital Twin is a dynamic, virtual replica of a physical object or process that is continuously updated with real-time sensor data. In industrial settings, this allows for predictive maintenance and system optimization without interfering with live operations.

In this project, Digital Twin technology is applied to home automation to bridge the gap between physical hardware and digital visualization. By streaming live data from the FPGA through a UART interface into a 3D model, the system creates a high-fidelity virtual home that mirrors the physical environment. This transforms raw sensor inputs into an interactive 3D dashboard, allowing users to intuitively monitor security states—such as fire detection or appliance status—in a photorealistic environment.

2.5 Blender as a Digital Twin Platform

Blender is an ideal platform for developing this project's Digital Twin due to its robust Python API (bpy), which enables the seamless manipulation of 3D objects via external data. Unlike heavy-duty game engines, which require significant overhead and runtime complexity, or web-based libraries that depend on complex browser frameworks, Blender offers a lightweight, high-fidelity environment. Its advanced rendering engine provides photorealistic visualization capabilities, while its Python-based scripting allows for precise, real-time updates of 3D scene elements. This combination makes Blender a powerful and efficient tool for bridging FPGA sensor logic with interactive, high-quality 3D monitoring.

2.6 UART-Based Monitoring Systems

UART (Universal Asynchronous Receiver-Transmitter) is a core communication protocol in embedded systems, widely favored for its simplicity and reliability in data transmission. By requiring only two signal lines—Transmit (TX) and Receive (RX)—UART provides an asynchronous, low-latency bridge between the FPGA and a PC. This protocol is particularly effective for low-bandwidth applications, such as transmitting system state updates. In this project, the UART interface acts as the vital link between the AMD Spartan-7 hardware and the Blender Digital Twin, ensuring that physical sensor triggers are reliably translated into virtual events with minimal overhead.

2.7 Research Gap

Existing home automation systems typically rely on microcontrollers that process sensor data sequentially. This architecture often leads to significant latency when multiple events occur simultaneously, making them less reliable safety. Furthermore, many current solutions are limited to single-purpose applications and lack advanced, real-time monitoring interfaces, often relying on basic status indicators that fail to provide users with comprehensive situational awareness.

The proposed system overcomes these limitations by utilizing the AMD Spartan-7 FPGA, which enables hardware-level parallel processing to monitor multiple sensors instantly. By implementing a priority-based Finite State Machine (FSM), the system ensures that critical hazards are handled without delay. Additionally, the integration of a Blender-based Digital Twin via UART communication provides a sophisticated visualization layer, effectively bridging the gap between raw hardware logic and intuitive, real-time home monitoring.

CHAPTER 3

METHODOLOGY

3.1 System Overview

The Home Automation and Safety System is built upon the AMD Spartan-7 FPGA, integrated into the RealDigital Boolean Board. This FPGA serves as the central processing unit, managing all sensor data and system logic. Developed using Verilog HDL and synthesized via the AMD Vivado Design Suite, the system architecture is designed for high-speed, parallel processing to ensure real-time environmental monitoring.

The system incorporates five core sensors—IR flame, MQ-2 gas, KY-028 temperature, LDR light, and HC-SR04 ultrasonic—to maintain comprehensive household safety. These inputs are processed by a hardware-level Finite State Machine (FSM) that evaluates conditions and triggers immediate local responses, such as buzzer alarms and LED visual indicators.

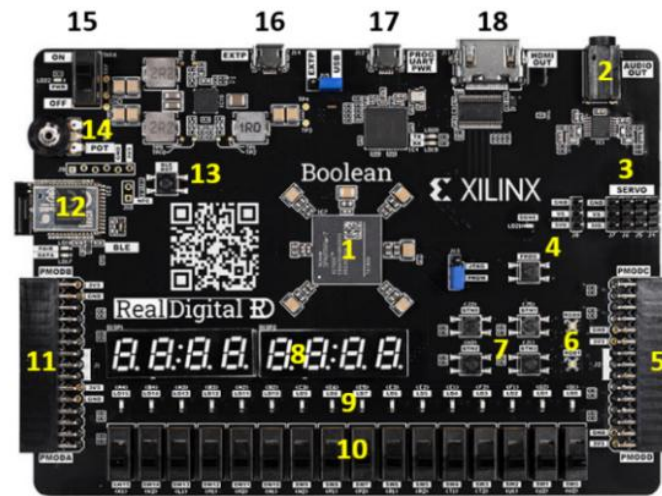
3.2 Design Goals

The developed system was designed with the following objectives:

- To continuously monitor multiple environmental and security conditions in real time.
- To achieve fast handling of all sensor inputs.
- To provide fast and reliable detection of hazardous conditions.
- To implement a priority-based Finite State Machine for system control.
- To generate immediate visual alerts through LEDs, RGB LED, and seven-segment display.
- To transmit the system status through UART for real-time visualization using a Blender-based digital twin.
- To develop a modular and scalable hardware design that can be easily expanded in future.

3.3 Hardware Components

3.3.1 Boolean Board (AMD Spartan-7 FPGA)



1	Spartan-7 XC7S50-CSGA324 FPGA	10	16 slide switches
2	Audio output	11	Expansion connector (30-pin)
3	Servo motor connectors (x4)	12	BLE radio module
4	FPGA Program button (reset)	13	BLE reset button
5	Expansion connector (30-pin)	14	10K Potentiometer (connected to ADC)
6	2 RGB LEDs	15	Power switch
7	4 pushbuttons	16	Accessory power connector
8	8-digit seven-segment display	17	Main USB (power, programming, UART)
9	16 discrete LEDs	18	HDMI source (up to 1080P)

Fig. 3.1: RealDigital Boolean Board (AMD Spartan-7 FPGA)

3.3.1.1 Spartan-7 FPGA Internal Architecture

The AMD Spartan-7 FPGA serves as the high-performance core of the home automation system. Unlike traditional microcontrollers that process instructions in a sequential "one-by-one" loop, the Spartan-7 utilizes a configurable hardware architecture that executes multiple logic tasks simultaneously. This parallel processing capability is the key to the system's speed, allowing it to monitor all five environmental sensors at once without the delays common in standard CPU-based systems.

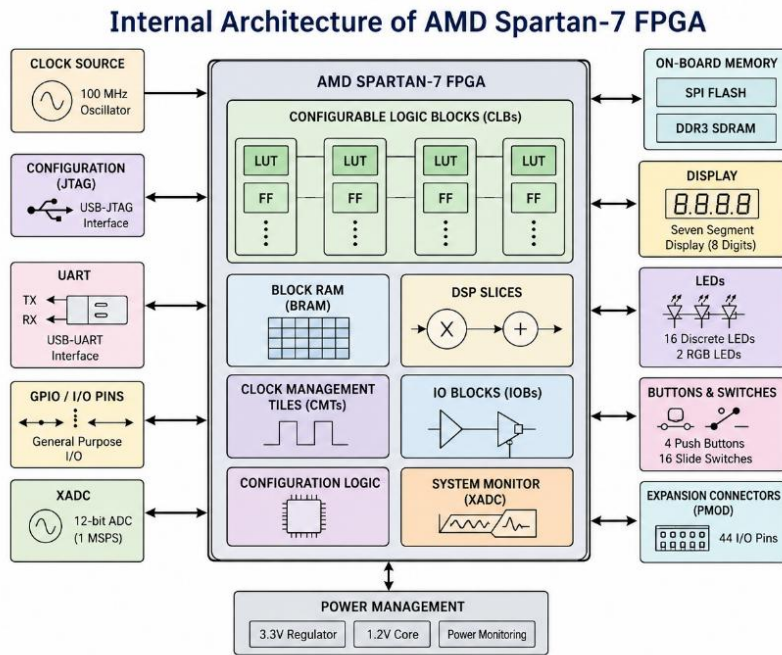


Fig. 3.2: Internal Architecture of the AMD Spartan-7 FPGA

Key Architectural Features:

- **Configurable Logic Blocks (CLBs) & LUTs:** These act as the "building blocks" of the system, implementing the custom Finite State Machine (FSM) that governs priority-based hazard detection.
- **GPIO Interface:** Provides direct, high-speed physical connections to the sensor suite (IR flame, MQ-2 gas, KY-028, LDR, and HC-SR04), ensuring raw data is captured with microsecond precision.
- **Clock Management Tiles (CMT):** Distributes the 100 MHz master clock throughout the device, keeping every logic module perfectly synchronized.
- **UART Communication:** A dedicated hardware serial interface that offloads system status data to the Blender Digital Twin, ensuring that the 3D visualization remains perfectly synced with the real-world sensor states.

3.3.1.2 Why AMD Spartan-7 FPGA was Selected

The AMD Spartan-7 FPGA was chosen for this project because it offers the speed, reliability, and flexibility required for real-time systems. Unlike standard microcontrollers that process tasks in a line, the Spartan-7 uses **hardware-level parallel processing**. This allows the system to monitor fire, gas, temperature, light, and motion sensors all at the same time, ensuring there is zero lag when an emergency occurs.

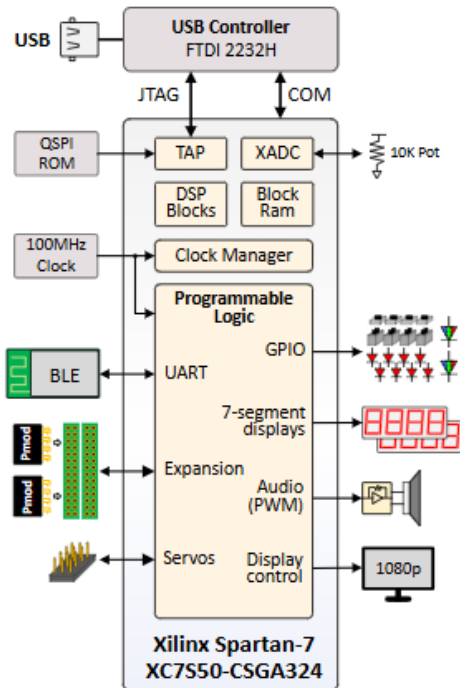


Fig. 3.3: Boolean Board Internal Block Diagram

Key Advantages for this Project:

- **Fast Response:** Because the logic is built directly into the hardware fabric, the system reacts to hazards instantly, far faster than traditional software-based controllers.
- **RealDigital Boolean Board Integration:** The board comes "ready to use" with all the necessary components—LEDs, RGB lights, 7-segment displays, and serial communication ports—already built-in. This saved us from needing extra circuit boards or complex external wiring.
- **Easy Programming:** Using the Vivado Design Suite, we could design our logic in Verilog, simulate it to ensure it worked, and then flash it onto the chip quickly.
- **Flexible & Scalable:** The board is highly modular. If we want to add more sensors, cameras, or advanced communication protocols in the future, we can simply update the FPGA design without having to rebuild the physical hardware.
- **Digital Twin Capability:** The onboard UART interface allows the FPGA to send real-time data to our 3D Blender model. This creates a seamless "Digital Twin," where the virtual house in Blender moves and updates instantly based on what the physical sensors on the Boolean Board are detecting.

Table 3.1: Specifications of the Boolean Board

Characteristic	Specification
FPGA Device	AMD (Xilinx) Spartan-7 XC7S50, part xc7s50csga324-1
Logic Cells	Approx. 52K logic cells (8,000+ slices, 4 LUTs + 8 flip-flops each)
DSP Slices	120 DSP blocks (dual 24-bit adders, 48-bit accumulator)
On-chip RAM	337 KB high-speed dual-port block RAM
ADC	12-bit, 1 MSPS analog-to-digital converter (XADC)
Clocking	5 Clock Management Tiles (CMTs)
On-board I/O	16 slide switches, 4 push buttons, 16 discrete LEDs, 2 RGB LEDs, two 4-digit 7-segment displays
Connectivity	HDMI out, 3.5 mm stereo audio out, 4 servo headers, optional BLE/Wi-Fi radio module
Expansion	PMODA / PMODB / PMODC / PMODD headers (44 additional FPGA I/O pins)
Programming Interface	Single USB port (power, JTAG programming, and USB-UART)
Design Software	AMD Vivado ML / Vivado WebPACK

3.3.2 HC-SR04 Ultrasonic Sensor

The HC-SR04 is an ultrasonic sensor used to measure the distance between the sensor and nearby objects. It has four pins: VCC, Trig, Echo, and GND. To start a measurement, the FPGA sends a short pulse to the Trig pin. The sensor then emits ultrasonic waves and waits for the reflected signal to return.



Fig. 3.4: HC-SR04 Ultrasonic Sensor Pinout

Table 3.2: Specifications of HC-SR04

Characteristic	Specification
Operating Voltage	5 V DC
Operating Current	< 15 mA
Measuring Range	2 cm – 400 cm
Accuracy	± 3 mm
Measuring Angle	15°
Trigger Signal	10 μ s TTL pulse
Echo Signal	TTL pulse, width proportional to distance

3.3.3 IR Flame Sensor

The IR flame sensor is used to detect the presence of fire by sensing the infrared radiation produced by a flame. It consists of three pins: VCC, OUT, and GND. Inside the module, an infrared receiver and a comparator circuit convert the detected infrared signal into a digital output.



Fig. 3.5: IR Flame Sensor Module

Table 3.3: Specifications of IR Flame Sensor Module

Characteristic	Specification
Operating Voltage	3.3 V – 5 V DC
Output Type	Digital (TTL), active-low
Detection Range	2 cm – 30 cm (adjustable via onboard potentiometer)

Response Time	< 2 ms
Comparator IC	LM393-type dual comparator

3.3.4 MQ Gas Sensor

The MQ gas sensor is used to detect harmful and combustible gases such as LPG, methane, carbon monoxide, and smoke. It works by changing the conductivity of its sensing material when exposed to these gases. The onboard comparator converts this change into a digital signal once the gas concentration crosses a preset threshold.



Fig. 3.6: MQ Gas Sensor Module

Table 3.4: Specifications of MQ Gas Sensor

Characteristic	Specification
Operating Voltage	5 V DC
Heater Resistance	$33\ \Omega \pm 5\%$
Heating Consumption	< 800 mW
Sensing Resistance	10 k Ω – 60 k Ω
Detection Range	200 – 10,000 ppm
Output Type	Digital (threshold via onboard comparator) and analog
Preheat Time	Over 10 minutes for stable analog readings

3.3.5 KY-028 Temperature Sensor

The KY-028 temperature sensor monitors the surrounding temperature using an NTC thermistor. As the temperature increases, the resistance of the thermistor decreases. The onboard comparator continuously compares this value with a preset threshold and changes the digital output whenever the threshold is exceeded.



Fig. 3.7: KY-028 Temperature Sensor Module

Table 3.5: Specifications of KY-028 Temperature Sensor

Characteristic	Specification
Operating Voltage	3.3 V – 5 V DC
Sensing Element	NTC Thermistor
Output Type	Digital (threshold) and analog
Threshold Adjustment	Onboard potentiometer
Response	Resistance decreases as temperature increases

3.3.6 LDR Light Sensor

The LDR (Light Dependent Resistor) sensor is used to detect the intensity of ambient light. Its resistance changes depending on the amount of light falling on the sensing element. An onboard comparator converts this change into a digital signal that can be easily processed by the FPGA.



Fig. 3.8: LDR Light Sensor Module

Table 3.6: Specifications of LDR Sensor Module

Characteristic	Specification
Operating Voltage	3.3 V – 5 V DC
Sensing Element	Cadmium Sulfide (CdS) photoresistor
Dark Resistance	In the order of mega-ohms
Bright-light Resistance	Few hundred ohms
Output Type	Digital (threshold via onboard comparator) and analog

3.4 Software Components

3.4.1 Vivado Design Suite:

This is the primary AMD software environment used for the entire FPGA development cycle, including Verilog HDL synthesis, simulation, implementation, and the generation of the bitstream file to program the Spartan-7 FPGA.

3.4.2 Blender:

Serving as the visualization engine, Blender provides the 3D environment for the Digital Twin. Its high-quality rendering capabilities allow for the photorealistic representation of the home, its appliances, and precaution indicators.

3.4.3 Python:

Integrated directly into Blender, Python scripts act as the middleware that monitors the computer's serial port, decodes incoming UART packets from the FPGA, and updates the 3D scene objects accordingly.

3.5 FPGA Hardware Design

3.5.1 Sensor Interface:

This module serves as the primary signal conditioning layer. It normalizes disparate sensor outputs—converting active-low signals (like the IR flame sensor) and active-high signals (like the LDR) into a unified, stable digital format. By cleaning up signal noise through debouncing logic, it ensures that the downstream FSM receives accurate, flicker-free inputs.

3.5.2 Ultrasonic Controller:

This module governs the HC-SR04 interface. It issues a precise 10 μ s trigger pulse and calculates distance by measuring the time-of-flight of the echo signal. By utilizing a counter synchronized to the 100MHz system clock, it achieves high precision in distance thresholding, allowing the system to reliably detect intruders or moving objects.

3.5.3 Sensor FSM:

The FSM is the central decision engine that interprets processed sensor data. It operates on a hard-coded hierarchy (Fire > Gas > Temp > Light > Object) to resolve priority conflicts. If multiple sensors are active, the FSM isolates the highest-priority threat, ensuring that the system output always reflects the most critical emergency condition first.

3.5.4 Seven Segment Controller:

To minimize pin usage, this controller employs time-division multiplexing. It rapidly cycles through each of the four digits at a frequency above the human persistence-of-vision threshold. This provides the illusion of a static, multi-character display while drastically reducing the hardware overhead on the Spartan-7's GPIO pins.

3.5.5 UART Transmitter:

This module acts as the bridge between parallel FPGA logic and serial communication. It buffers the current state data and shifts it out bit-by-bit according to standard UART timing specifications, ensuring that status packets are ready for transmission to external systems.

3.6 Software Design

The system is developed using a modular Verilog HDL design, which simplifies debugging, verification, and future scalability. All hardware logic is synthesized and tested within the AMD Vivado Design Suite before integration into the top-level module. The architecture comprises a Sensor Interface Module for signal conditioning, an Ultrasonic Controller for precise distance measurement, and a central Finite State Machine (FSM) that processes inputs based on a predefined priority logic to determine system status.

Output control logic manages real-time feedback through onboard LEDs, an RGB LED, and a seven-segment display, while the UART Communication Module broadcasts status codes

at 9600 bps to the Blender Digital Twin. This modular approach ensures that individual functional blocks remain independent and reliable, allowing for future system expansion or the seamless integration of additional sensors without requiring extensive redesign.

3.7 UART Communication Design

3.7.1 Protocol:

The system utilizes a standard asynchronous serial protocol configured for 9600 baud, 8 data bits, no parity, and 1 stop bit. This configuration was selected for its universal compatibility with PC serial ports, ensuring reliable, low-overhead communication between the Spartan-7 board and the host computer.

3.7.2 Frame Format:

Data is encapsulated in a standard frame consisting of a single start bit, 8 data bits representing the message payload, and a stop bit. This structure provides the necessary synchronization for the receiver to accurately capture each byte without requiring a shared clock line.

3.7.3 Message Format:

Communication is text-based to ensure human-readability during debugging. The system sends a 5-character fixed-length string (e.g., FgTLd). Each character slot corresponds to a specific sensor; if a character is present (e.g., 'F'), the sensor is triggered, while a null or placeholder character indicates a safe state.

3.8 Blender Digital Twin Design

3.8.1 House Modelling:

The house was constructed using a low-poly workflow to ensure the scene remains lightweight for real-time interaction. Each structural component—such as walls, roofs, and doors—is modeled as a distinct mesh, which allows for efficient updates without putting undue stress on the Blender render engine.

3.8.2 Object Hierarchy:

The scene utilizes a structured 'Parent-Child' hierarchy. By grouping all sensors and appliances under a master "House" object, the Python script can use relative pathing to target

specific scene elements, which simplifies the object-lookup logic and keeps the script clean and scalable.

3.8.3 Materials & Lighting:

Lighting and material dynamics are driven by Emission shaders. By keyframing the "Emission Strength" parameter, we allow materials to glow when an alert state is active. This provides a clear, unmistakable visual signal in the 3D space, mimicking the behavior of physical alarm indicators.

3.8.4 Dashboard Design:

The dashboard is a 3D interface integrated into the viewport. It displays dynamic text that corresponds to the FSM state. This HUD acts as an "At-a-glance" status panel, effectively consolidating complex sensor telemetry into simple, readable text that users can check instantly within the 3D scene.

3.8.5 Animation Design:

Animations are driven by stored "Actions" in Blender. The Python script triggers these actions—such as the rotation of a virtual fan or the sliding of a door—using keyframe interpolation. This creates a fluid, immersive experience where the virtual house physically reacts to the sensor data received from the FPGA.

3.9 Blender Python Design

3.9.1 UART Reception:

Using the pyserial library, this module opens a non-blocking connection to the system's virtual COM port. The script is configured to listen for the incoming 5-character packets, employing a buffer that clears after every successful read to prevent "lag" caused by stale data buildup.

3.9.2 State Decoding:

This logic maps each incoming character to a specific Boolean flag in the scene. The decoder evaluates the packet index by index; for instance, index 0 is checked for 'F'. If 'F' is found, it triggers the "Fire Alarm" function, which updates the relevant material indices and object visibilities.

3.9.3 Object Synchronization:

Synchronization is achieved by calling `bpy.data.objects`. The script iterates through the scene, updating object properties like `.hide_viewport` for security doors or changing material emission levels for alarm lamps. This loop ensures that the visual representation in Blender is a true, real-time reflection of the FPGA hardware state.

3.9.4 Real-Time Update Timer:

The `bpy.app.handlers.frame_change_pre` handler is the "heartbeat" of the integration. By tying the Python execution to Blender's frame refresh cycle, the script runs continuously at the scene's frame rate. This ensures that the moment a packet arrives, the 3D environment updates to reflect the new reality without noticeable delay.

3.10 Functional Test Cases

To verify the functionality of the proposed Home Automation and Security System, a series of simulation and hardware tests were conducted under different operating conditions. Each test case evaluated the response of the FPGA to specific sensor inputs and verified the corresponding outputs generated by the system.

Table 3.7 Functional Test Cases

Test Case	Input Condition
Reset	Reset Button Pressed
Fire Detection	IR Sensor Active
Gas Detection	MQ Sensor Active
Temperature Detection	KY-028 Active
Light Detection	LDR Active
Object Detection	Ultrasonic Detects Object

Table 3.8 Expected vs Actual Results

Function	Expected Result
Fire Detection	Correct alert generation
Gas Detection	Correct gas indication
Temperature Detection	Display 't'
Light Detection	Display 'L'
Object Detection	Display 'd'
UART Communication	Correct status transmission

CHAPTER 4

IMPLEMENTATION

4.1 Development Environment

The AMD Vivado Design Suite was used for hardware design, simulation, synthesis, implementation, and FPGA programming, while Blender (with Python scripting) was used to develop an interactive 3D digital twin for real-time visualization through UART communication.

The development environment was selected to support the complete FPGA design flow, from Verilog coding to hardware verification on the RealDigital Boolean Board

4.2 Vivado Design Flow

After developing the hardware modules in Verilog HDL, the design was verified through behavioral simulation, synthesized into hardware logic, implemented on the target FPGA device, and finally converted into a configuration bitstream for programming the RealDigital Boolean Board. Each stage of the design flow is described in the following sections.

4.2.1 Behavioral Simulation

Behavioral simulation was performed to verify the proposed design before FPGA implementation. A Verilog testbench was used to apply different sensor conditions and observe the generated outputs.

The simulation verified the operation of the Ultrasonic Controller, Sensor FSM, Seven-Segment Display Controller, UART Communication Module, and Top-Level Module under various operating conditions. The results confirmed that the system responded correctly before synthesis and hardware implementation.

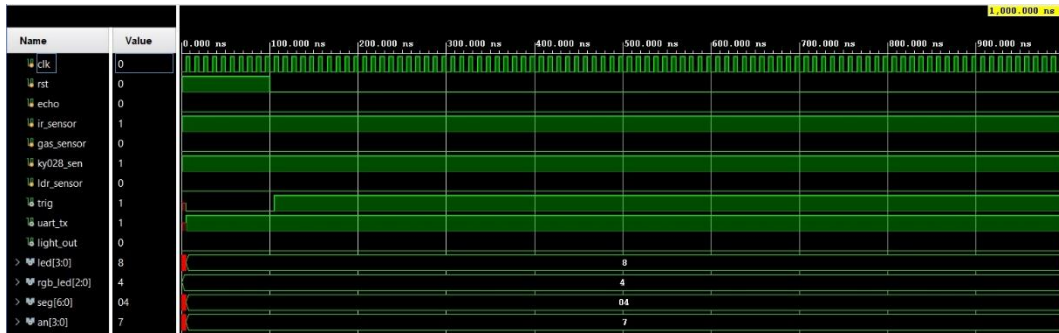


Fig. 4.1: Behavioral Simulation

4.2.2 RTL Analysis

After successful behavioral simulation, the design was synthesized to generate the Register Transfer Level (RTL) representation. The RTL schematic shows the hardware modules and their interconnections, confirming that the Top-Level Module correctly integrates the Ultrasonic Controller, Sensor FSM, Seven-Segment Display Controller, UART Communication Module, and supporting logic.

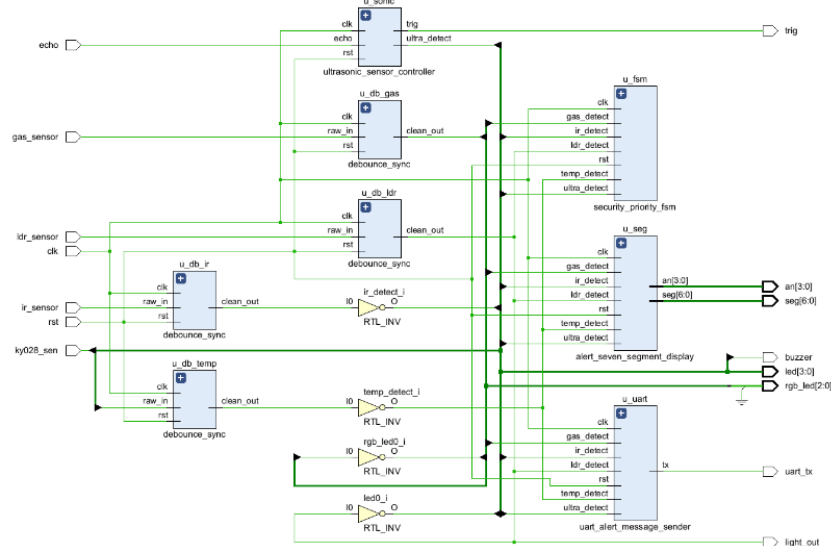


Fig. 4.2: RTL Analysis

RTL analysis verifies that the hardware architecture generated by Vivado matches the intended Verilog HDL design before FPGA implementation.

4.2.3 Synthesis

Synthesis converts the Verilog HDL description into a technology-specific digital circuit that can be implemented on the AMD Spartan-7 FPGA. During this process, Vivado translates the HDL code into logic gates, flip-flops, multiplexers, and other FPGA resources while performing logic optimization to improve hardware efficiency.

The synthesis report provides information about the FPGA resources utilized by the proposed design, including Look-Up Tables (LUTs), Flip-Flops (FFs), Input/Output (I/O) pins, Block RAM (BRAM), and DSP slices. It also reports any synthesis warnings or optimization messages. Successful synthesis confirms that the hardware design can be mapped onto the selected FPGA device.

```
-----
Start Writing Synthesis Report
-----

Report BlackBoxes:
+-----+
| BlackBox name |Instances |
+-----+
+-----+

Report Cell Usage:
+-----+
| Cell |Count |
+-----+
|1| |BUFG | 1|
|2| |CARRY4 | 33|
|3| |LUT1 | 11|
|4| |LUT2 | 13|
|5| |LUT3 | 13|
|6| |LUT4 | 30|
|7| |LUT5 | 41|
|8| |LUT6 | 47|
|9| |MUXF7 | 3|
|10| |FDSR | 173|
|11| |FDSR | 7|
|12| |IOBUF | 7|
|13| |IOBUF | 22|
+-----+

Finished Writing Synthesis Report : Time (s): cpu = 00:01:01 ; elapsed = 00:01:08 . Memory (MB): peak = 1835.734 ; gain = 1125.770
-----
```

Fig. 4.3: Synthesis Report

The synthesis report verifies that the proposed design has been successfully translated into FPGA hardware logic and is ready for implementation.

4.2.4 Implementation

Implementation is the stage in which the synthesized logic is physically mapped onto the FPGA device. During this process, Vivado performs placement and routing, assigning each logic element to a specific hardware resource within the AMD Spartan-7 FPGA while establishing the required signal connections.

The implementation stage also performs timing analysis to ensure that the design satisfies all setup and hold time requirements for the specified 100 MHz system clock. Successful implementation confirms that the design can operate reliably without timing violations.

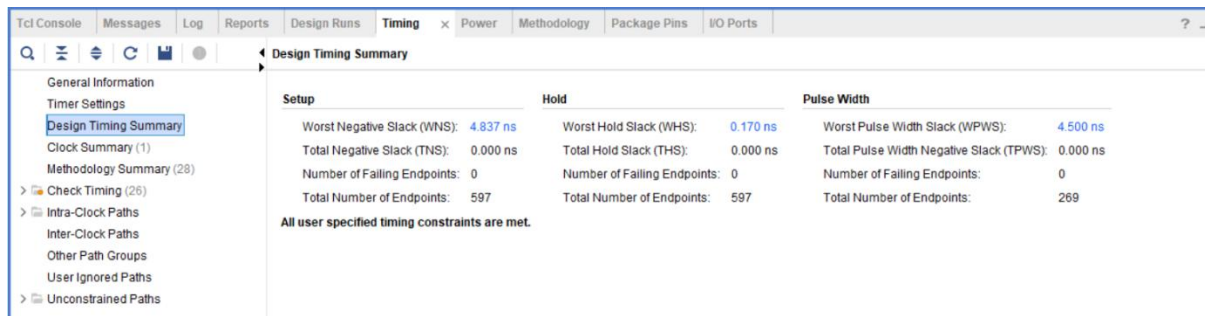


Fig. 4.4: Implementation Timing Report

The implementation report confirms that the synthesized hardware has been successfully placed and routed on the FPGA while satisfying the specified timing constraints.

4.2.5 Bitstream Generation

After successful implementation, Vivado generates the FPGA configuration bitstream. The bitstream is a binary configuration file that programs the internal logic resources of the AMD Spartan-7 FPGA according to the developed hardware design.

The generated bitstream contains the complete configuration required to implement the proposed Home Automation and Security System on the RealDigital Boolean Board. This file is later downloaded to the FPGA using the Vivado Hardware Manager.

Successful bitstream generation indicates that the design has completed all stages of the FPGA development flow and is ready for hardware programming.

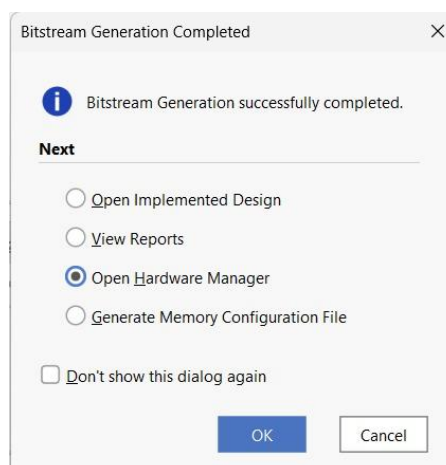


Fig. 4.5: Bitstream Generated Successfully

The successful generation of the bitstream confirms that the proposed FPGA design is complete and ready to be programmed onto the target hardware platform.

4.3 FPGA Programming on the RealDigital Boolean Board

After generating the configuration bitstream, the design was programmed onto the AMD Spartan-7 FPGA on the RealDigital Boolean Board using the Vivado Hardware Manager. The board was connected through a USB cable for JTAG programming and UART communication. After successful programming, the FPGA initialized all modules and began monitoring the connected sensors, updating the output devices, and transmitting the system status to the Blender Digital Twin.

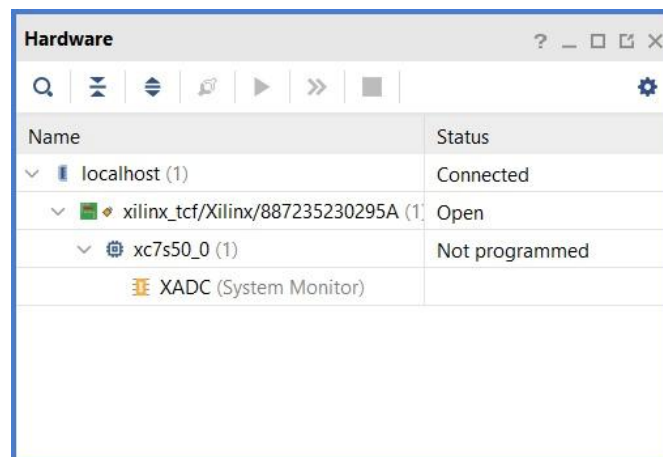


Fig. 4.6: Vivado Hardware Manager

The Vivado Hardware Manager detects the target FPGA through the JTAG interface and is used to download the generated bitstream.

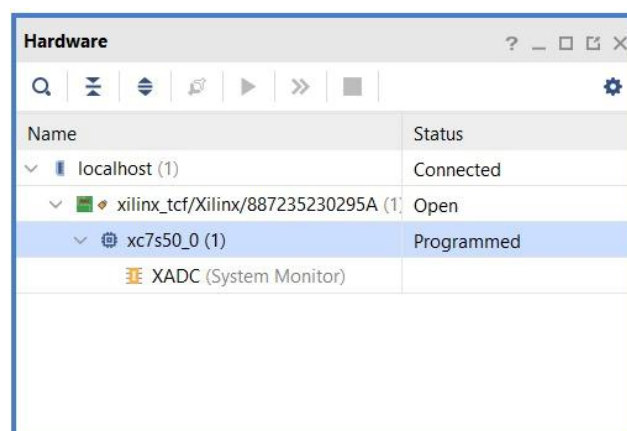


Fig. 4.7: FPGA Programmed Successfully

The successful programming message confirms that the bitstream has been downloaded correctly and the FPGA is ready to execute the proposed Home Automation System.

4.3 Blender Implementation

4.3.1 House Creation:

The virtual house was modeled using a low-poly architectural approach in Blender to ensure smooth, real-time performance. Key structural elements—such as doors, windows, and light fixtures—were designed as individual objects, allowing the Python middleware to manipulate them independently. This object-oriented modeling is crucial for maintaining a clean hierarchy that can be easily mapped to specific sensor states transmitted from the FPGA.

4.3.2 Materials:

High-fidelity materials were applied to the 3D scene to differentiate between safe and hazardous states. We utilized Principled BSDF shaders with emission properties; for instance, the fireplace or light fixture materials remain neutral during normal operation but shift to an intense "emissive red" when the FPGA signals a fire or hazard. This visual transition provides an immediate, intuitive cue to the user regarding the current safety status of the house.

4.3.3 Lighting:

Scene lighting was configured to dynamically respond to the system state, creating an immersive atmosphere. The environment utilizes a mix of ambient world lighting and localized point lights tied to the virtual light fixtures. When the LDR sensor detects low light, the virtual scene's interior lights are programmed to automatically toggle on, demonstrating the home automation capabilities of the system within the digital twin.

4.3.4 Dashboard:

A dedicated 3D heads-up display (HUD) was constructed within the Blender viewport to provide a clear summary of system telemetry. This dashboard features dynamic text labels that update in real-time, displaying specific sensor status messages such as "FIRE ALERT" or "GAS LEAK DETECTED." By integrating this dashboard directly into the 3D render, users can monitor the system's overall health without needing to toggle between different application windows.

4.3.5 Animation:

To bring the house to life, we created specific animation clips for interactive elements, such as door openings, fan rotations, and light fading. These animations are controlled by Python keyframe insertion logic, which triggers the stored movements the instant a corresponding serial packet is received. This adds a layer of kinetic feedback that confirms the system is not only monitoring data but is also actively responding to environmental changes.

4.4 Python Script Implementation

The Python script functions as the critical middleware bridge between the physical hardware and the Blender environment. Utilizing the pyserial library, the script continuously monitors the USB-UART COM port in the background, waiting for the 5-character state packet sent by the FPGA. Upon receiving a packet, the script executes a decoding function that maps each character to a specific object property—such as toggling an object's visibility or changing its material index. By registering this script with Blender's `frame_change_pre` handler, the logic runs on every frame render, ensuring that the virtual twin remains perfectly synchronized with the FPGA's real-time processing loop.

4.4.1 Serial Communication:

This module establishes a persistent, low-latency connection between the workstation and the FPGA. Using the pyserial library, the script is configured to match the FPGA's transmission settings (9600 baud, 8N1). By implementing non-blocking read calls, the script ensures that the Blender UI remains responsive, preventing the 3D viewport from freezing while waiting for incoming data from the physical hardware.

4.4.2 UART Parser:

Once data is received, the parser processes the raw incoming byte stream into a structured 5-character string. The parser includes a validation step that checks for the correct message length and packet structure before passing the data to the logic controller. This error-handling layer ensures that transient signal noise or partial packets do not trigger false state changes in the virtual house.

4.4.3 Digital Twin Update Logic:

This is the primary control loop that translates decoded characters into 3D scene manipulations. It maps each character (e.g., 'F' for Fire) to a specific Python function call that

modifies the properties of Blender objects. Whether it is toggling the `hide_viewport` property of an alarm object or updating the color of an emissive material, this logic ensures that the physical state of the Spartan-7 hardware is rendered in the virtual environment within milliseconds.

4.5 Challenges During Implementation

- **Sensor Polarity:** Mismatched active-high and active-low signals led to inconsistent data. This was resolved by implementing logic inversion within the FPGA to ensure all sensor inputs were uniform.
- **Ultrasonic Timing:** Initial distance measurements were unstable due to timing inaccuracies. We refined the trigger pulse and echo duration measurements using the system's 100 MHz clock to ensure precise object detection.
- **UART Synchronization:** Mismatched baud rates caused data corruption between the FPGA and Blender. Setting a fixed 9600 bps rate on both ends restored reliable communication.
- **Seven-Segment Multiplexing:** Low refresh rates caused flickering. We optimized the multiplexing frequency to maintain steady character display while minimizing FPGA resource usage.
- **XDC Pin Mapping:** Incorrect physical pin assignments initially caused hardware communication failures. These were resolved by cross-referencing the RealDigital Boolean Board schematic and updating the constraint file.
- **Hardware Debugging:** To manage integration complexity, we verified each module individually through simulation and physical testing (LEDs, UART logs) before final system integration, ensuring stable operation.

CHAPTER 5

RESULTS

5.1 Test Environment

Behavioral simulation was performed using the AMD Vivado Design Suite to validate the Verilog HDL design before FPGA implementation. After successful simulation, the generated bitstream was programmed onto the AMD Spartan-7 FPGA on the RealDigital Boolean Board for hardware validation.

5.2 Hardware Validation

Each sensor was tested individually, and the corresponding outputs were observed through simulation waveforms, onboard LEDs, RGB LED, seven-segment display, and the Blender digital twin. The hardware responses matched the simulation results, confirming reliable operation of the FPGA-based system.

5.2.1 Reset Verification

The reset signal was applied to initialize the system before normal operation. After reset, the Sensor Finite State Machine (FSM) entered the Normal state, all outputs returned to their default conditions, and the Blender digital twin displayed the normal system status. This verified proper system initialization and stable startup operation.

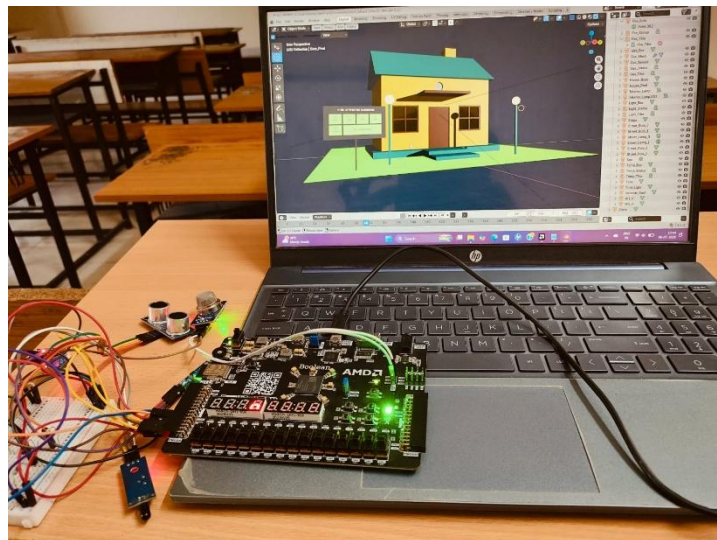


Fig. 5.1: Reset Verification (Hardware and Blender Output)

5.2.2 Fire Detection

The IR flame sensor was activated to simulate a fire condition. The Sensor FSM immediately entered the Fire state. The corresponding LED was turned ON, the seven-segment display showed F, and the UART module transmitted the fire status to the Blender digital twin, where the fire alert was displayed. The hardware response matched the simulation results, confirming correct implementation of the fire detection module.

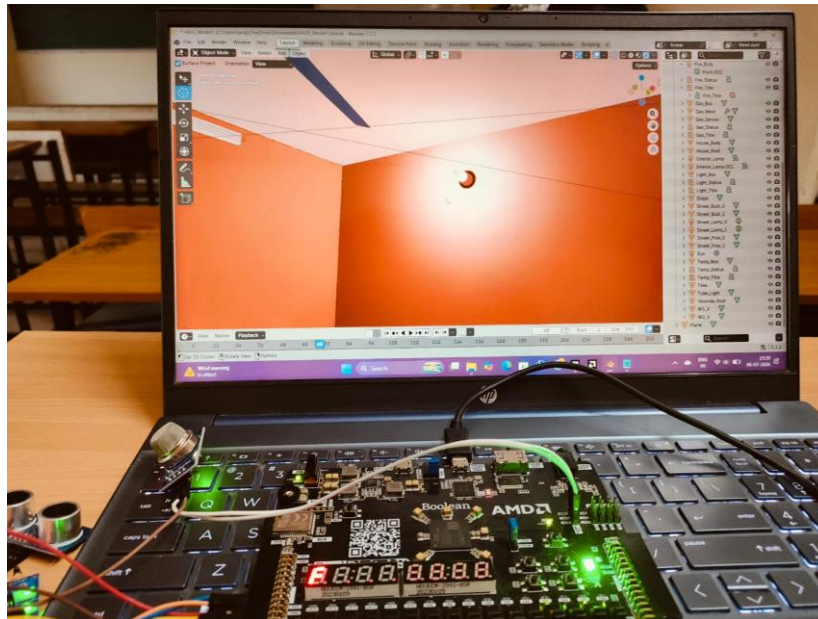


Fig. 5.2: Fire Detection (Hardware and Blender Output)

5.2.3 Gas Detection

The MQ gas sensor was tested by exposing it to combustible gas. When the gas concentration exceeded the threshold, the FSM entered the Gas state. The corresponding LED and RGB LED were activated, the seven-segment display displayed g, and the gas status was transmitted through UART to the Blender digital twin. The observed hardware outputs agreed with the simulation results.

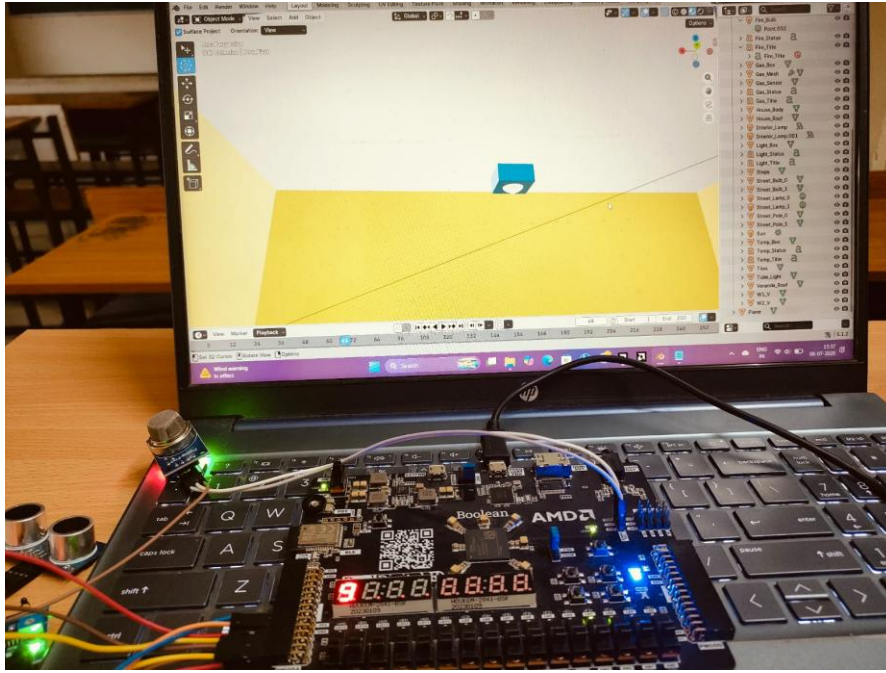


Fig. 5.3: Gas Detection (Hardware and Blender Output)

5.2.4 Temperature Detection

The KY-028 temperature sensor was tested by increasing the surrounding temperature above the predefined threshold. The Sensor FSM changed to the **Temperature** state, the corresponding LED was activated, and the seven-segment display displayed **t**. The UART module successfully transmitted the updated status to the Blender digital twin. The hardware validation confirmed the correct operation of the temperature monitoring system.

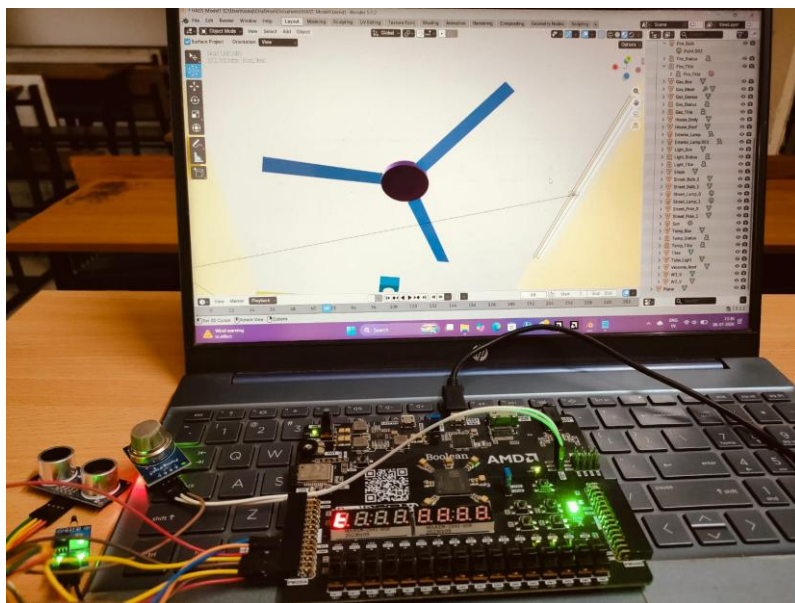


Fig. 5.4: Temperature Detection (Hardware and Blender Output)

5.2.5 Light Detection

The LDR sensor was tested by reducing the ambient light intensity below the configured threshold. The Sensor FSM entered the Light Detection state, the corresponding output indicator was activated, and the seven-segment display showed L. The updated status was transmitted to the Blender digital twin through the UART interface. The hardware behavior matched the simulation results.

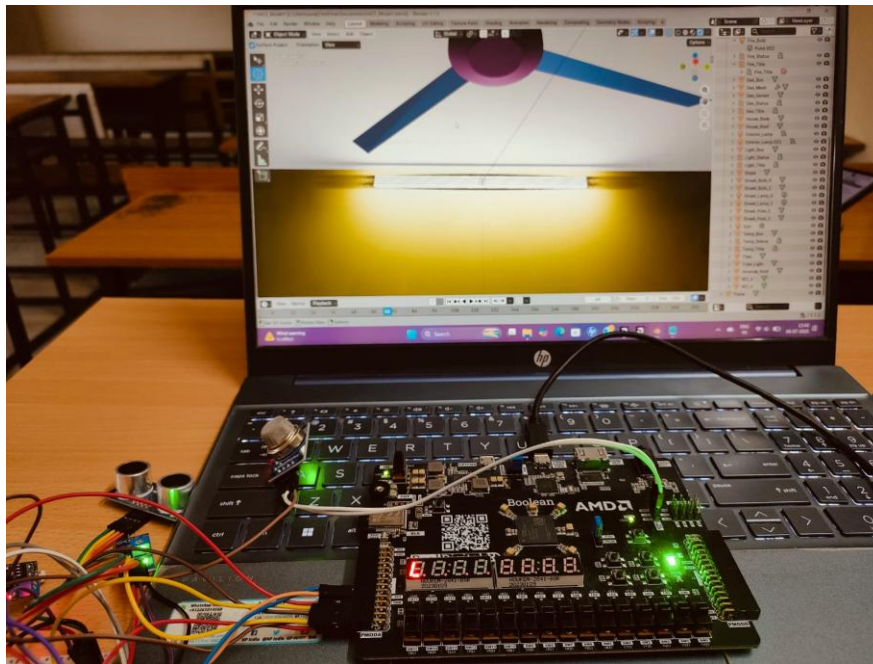


Fig. 5.5: Light Detection (Hardware and Blender Output)

5.2.6 Object Detection

The HC-SR04 ultrasonic sensor was tested by placing an object within the configured detection range. The Ultrasonic Controller generated the trigger pulse, measured the echo duration, and produced the `ultra_detect` signal. The Sensor FSM entered the Object Detection state, the seven-segment display displayed d, and the UART module transmitted the object detection status to the Blender digital twin. The simulation and hardware results confirmed accurate and reliable object detection.

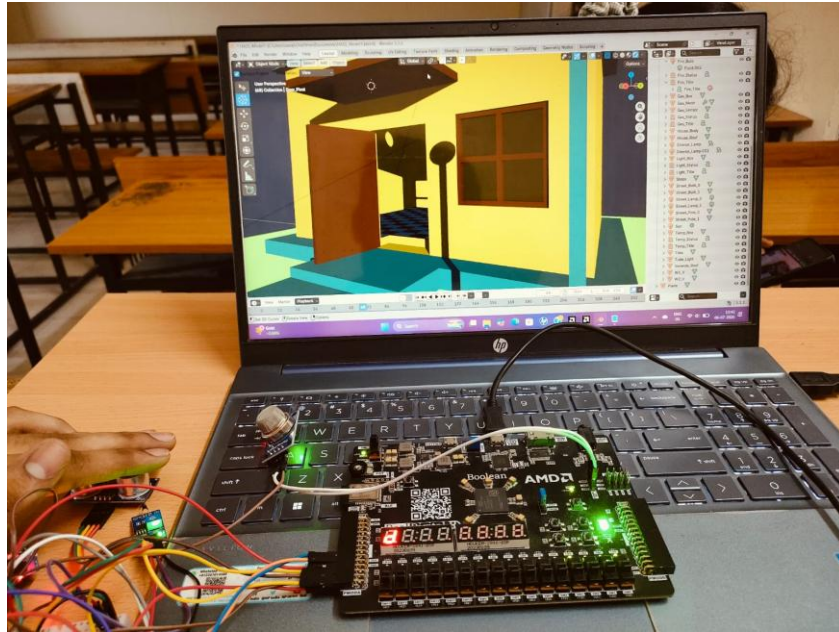


Fig. 5.6: Object Detection (Hardware and Blender Output)

5.3 UART Validation

The UART communication was validated by conducting extensive stress tests on the data transmission bridge between the FPGA and the host PC. Using a serial terminal monitor, we verified that the FPGA consistently transmitted the correct 5-character state packet at the defined 9600-baud rate, regardless of sensor trigger frequency. We performed "boundary testing" by forcing multiple sensor states simultaneously to ensure the packet structure remained intact and that the serial buffer did not overflow or drop bits during high-speed transitions. This validation step confirmed that the data arriving at the workstation was a stable and accurate reflection of the FPGA's internal Finite State Machine (FSM) state, providing a reliable foundation for the Digital Twin.

5.4 Blender Validation

Blender validation involved testing the responsiveness and synchronization of the 3D scene against known hardware inputs. We utilized a "Stimulus-Response" methodology, where controlled physical triggers (such as applying controlled heat to the temperature sensor or triggering the IR flame sensor) were performed while observing the Digital Twin's visual output. We measured the "Time-to-Visualization," verifying that the latency between the physical sensor event and the corresponding update in the 3D scene (e.g., the activation of a virtual fire alarm) remained within sub-second thresholds. Furthermore, we verified that the Python middleware correctly handled state reversals—ensuring that virtual objects correctly

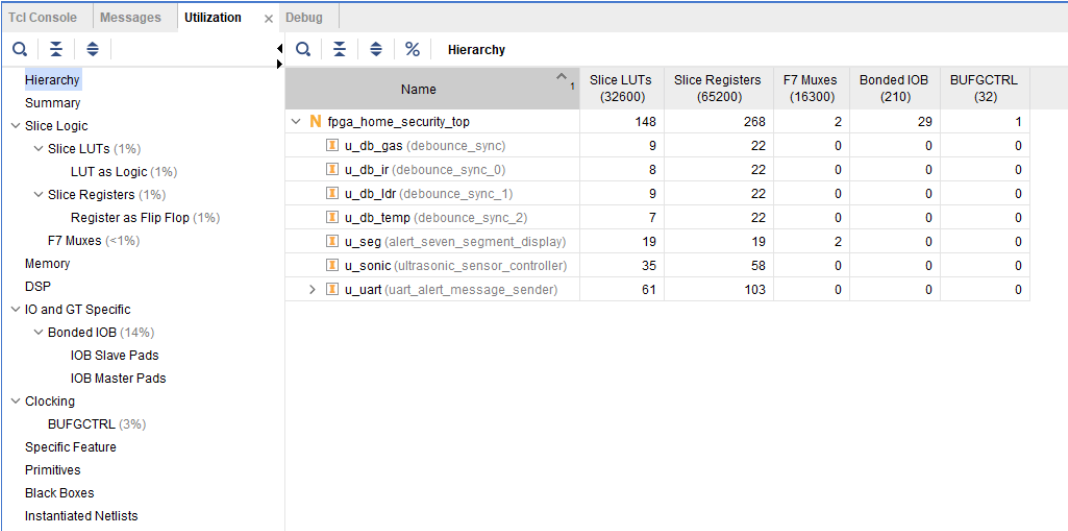
reset to their "safe" states immediately after physical sensors were deactivated—thereby confirming that the twin maintained perfect temporal synchronization with the physical hardware.

5.5 FPGA Results

The FPGA resource utilization report generated by the AMD Vivado Design Suite provides information about the hardware resources consumed by the proposed design after synthesis and implementation. The report indicates the number of Look-Up Tables (LUTs), Flip-Flops (FFs), Input/Output (I/O) pins, Block RAM (BRAM), and other FPGA resources used during implementation.

The proposed system uses only a small portion of the available resources on the AMD Spartan-7 FPGA. This demonstrates that the implemented design is resource-efficient and leaves sufficient hardware resources available for future expansion, such as additional sensors, wireless communication modules, or advanced processing algorithms.

The moderate utilization of FPGA resources also contributes to simplified routing, improved timing performance, and reliable hardware operation.



Name	Slice LUTs (32600)	Slice Registers (65200)	F7 Muxes (16300)	Bonded IOB (210)	BUFGCTRL (32)
fpga_home_security_top	148	268	2	29	1
u_db_gas (debounce_sync)	9	22	0	0	0
u_db_ir (debounce_sync_0)	8	22	0	0	0
u_db_ldr (debounce_sync_1)	9	22	0	0	0
u_db_temp (debounce_sync_2)	7	22	0	0	0
u_seg (alert_seven_segment_display)	19	19	2	0	0
u_sonic (ultrasonic_sensor_controller)	35	58	0	0	0
u_uart (uart_alert_message_sender)	61	103	0	0	0

Fig. 5.7: FPGA Resource Utilization Report

5.5.1 Timing Analysis

The timing analysis report generated by the AMD Vivado Design Suite was used to verify that the proposed hardware design satisfies the required timing constraints. During implementation, Vivado performed static timing analysis to evaluate the propagation delays, clock distribution, setup time, and hold time requirements of the implemented FPGA design.

The analysis confirmed that the design successfully met the timing requirements of the 100 MHz system clock without any setup or hold time violations. The successful timing closure indicates that the implemented hardware can operate reliably at the specified clock frequency while maintaining stable communication between all functional modules.

Meeting the timing constraints is an important indicator of design reliability because it ensures that data is transferred correctly between sequential logic elements during every clock cycle.

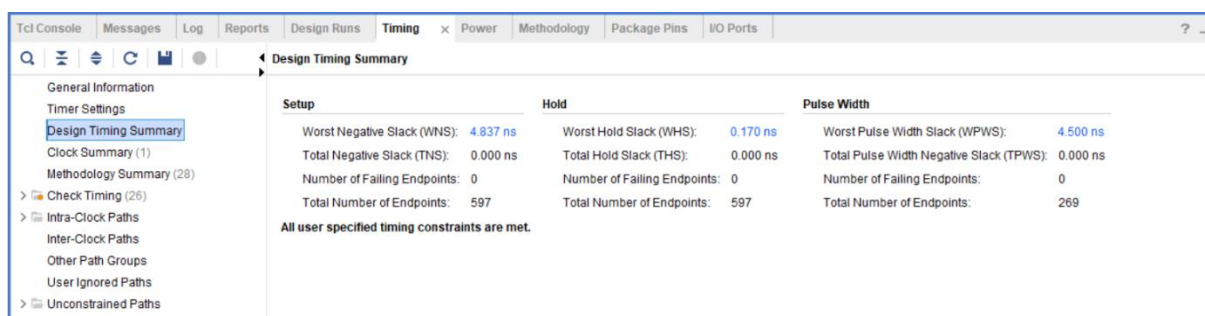


Fig. 5.8: Timing Summary Report

5.6 UART Results

This section presents the validation of the UART communication link between the AMD Spartan-7 FPGA and the host computer. The objective was to ensure that the internal system state—determined by the priority-based FSM—is transmitted accurately and interpreted correctly by the receiving Digital Twin middleware.

- **Data Integrity and Packet Consistency:** Oscilloscope and terminal monitoring confirmed that the serial signal maintains the defined 9600 baud rate with high signal integrity, ensuring that the 5-character state packets (e.g., FgTLd) are delivered without frame errors or bit-flipping.
- **Figure Reference [Insert your photo here]:** The following image displays the real-time serial output monitored via a terminal software, demonstrating the consistency of the transmitted string FgTLd in response to varying sensor triggers on the hardware board.

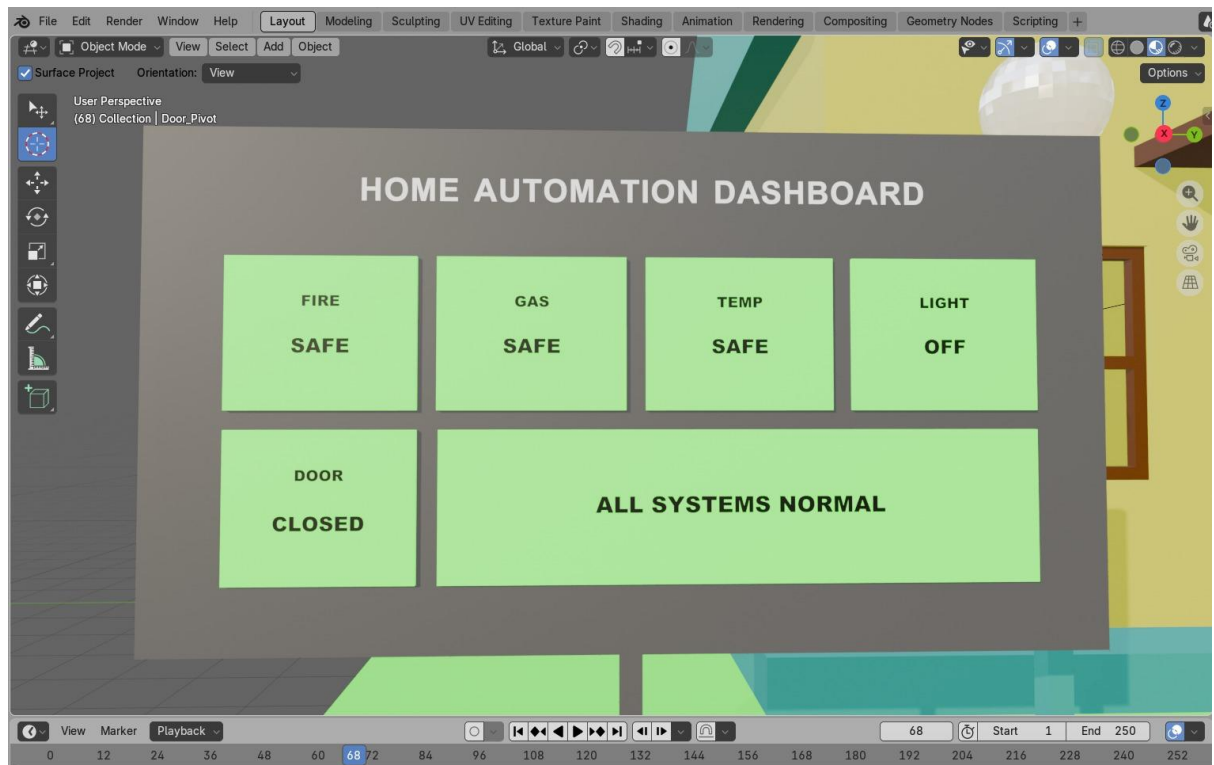


Fig. 7.9: UART RESULTS

5.7 Blender Digital Twin Results

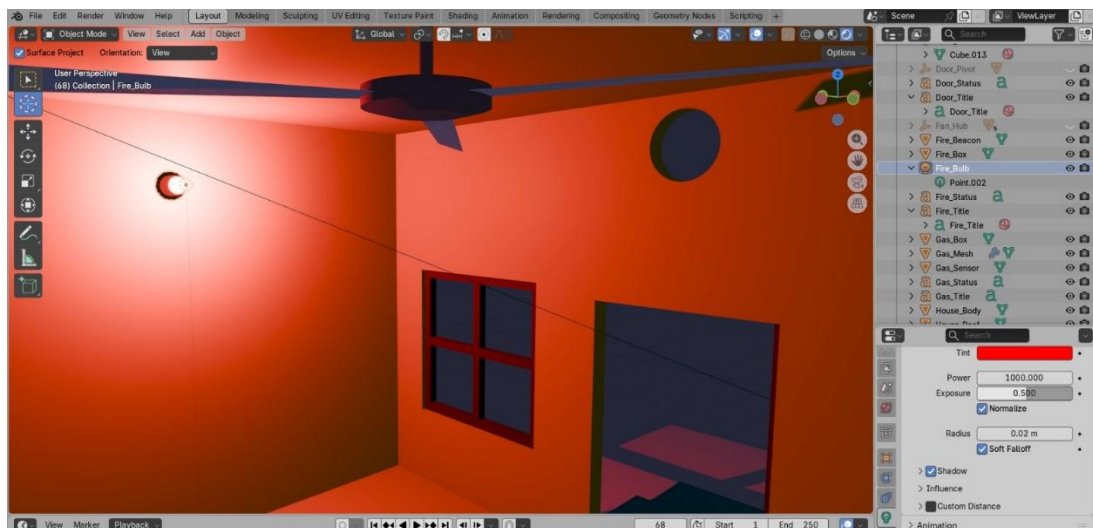


Fig. 7.10: Fire Alarm Visual in Blender

The Blender Digital Twin correctly displays the fire alarm whenever the IR flame sensor detects a fire condition. The virtual environment immediately reflects the hardware state by activating the corresponding fire indication. This demonstrates successful synchronization between the FPGA and the Digital Twin through UART communication.

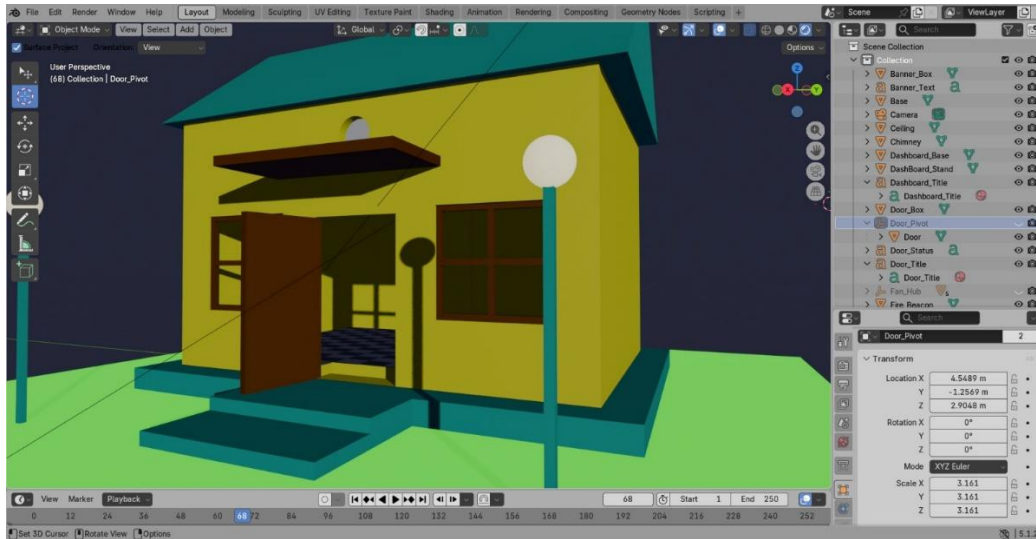


Fig. 7.11: Door Visual in Blender

When the ultrasonic sensor detects an object within the predefined range, the Digital Twin updates the door visualization accordingly. The virtual door animation represents the real-time hardware status with minimal delay. This confirms accurate object detection and proper communication between the FPGA and Blender



Fig. 7.12: Light Visual in Blender

The Digital Twin automatically updates the lighting condition based on the output of the LDR sensor. When low light is detected, the virtual light turns ON to match the hardware response. This demonstrates the successful implementation of the home automation feature within the 3D environment.

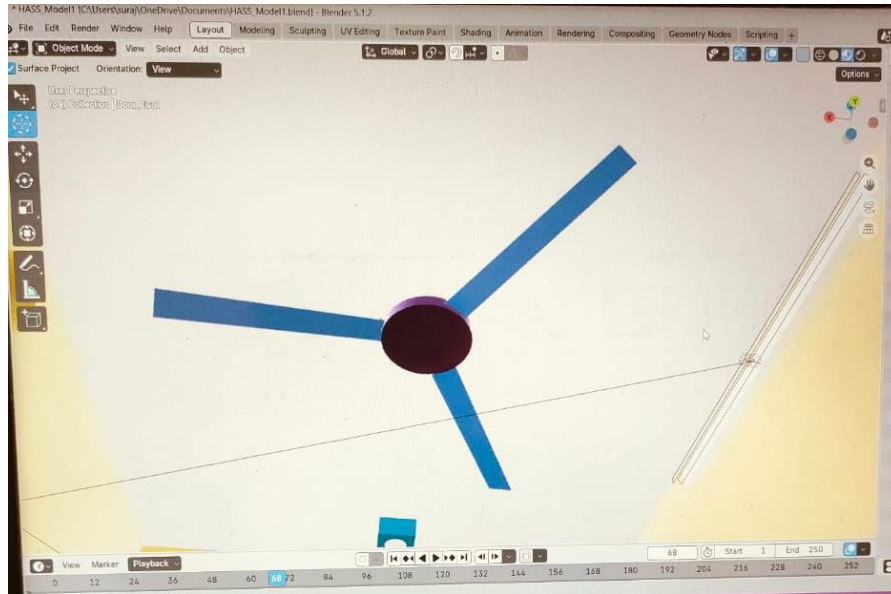


Fig. 7.13: Temperature Visual in Blender

The temperature visualization changes immediately when the KY-028 sensor detects a temperature above the preset threshold. The Blender scene accurately reflects the updated system status received from the FPGA. This confirms reliable temperature monitoring and real-time visualization.

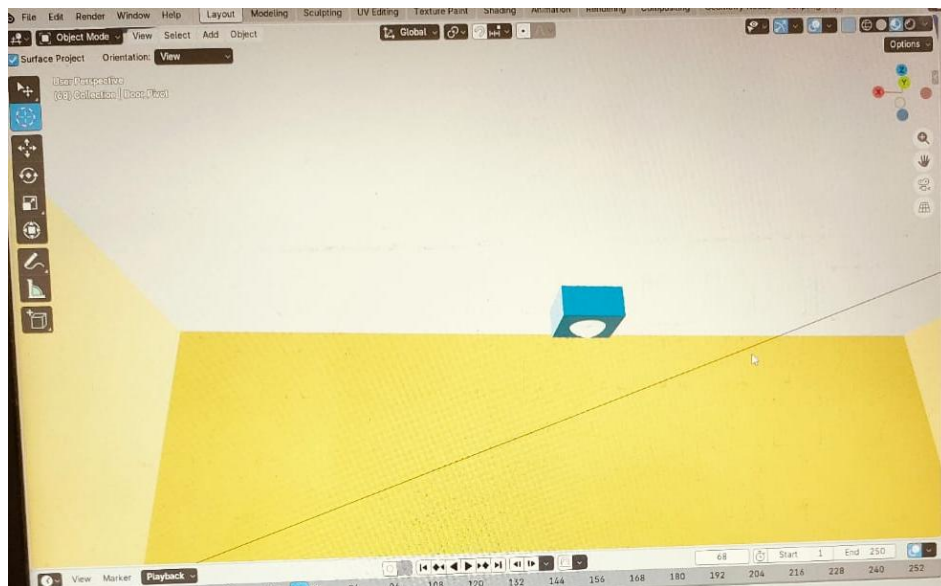


Fig. 7.14: Gas Sensor Visual in Blender

When the MQ gas sensor detects gas leakage, the Digital Twin activates the corresponding gas warning indication. The visualization matches the hardware outputs and UART messages generated by the FPGA.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Project Achievements

The proposed project successfully achieved the objectives defined during the design phase. The major achievements are summarized below.

- Successfully designed and implemented a Home Automation System using the AMD Spartan-7 FPGA.
- Integrated five different sensors for monitoring fire, gas leakage, temperature, light intensity, and object detection.
- Developed the complete digital hardware using Verilog HDL following a modular design approach.
- Implemented a priority-based Finite State Machine (FSM) for intelligent decision-making.
- Generated real-time visual indications using onboard LEDs, RGB LED, and a four-digit seven-segment display. .
- Established reliable UART communication between the FPGA and a Blender Digital Twin.
- Successfully developed a graphical dashboard capable of displaying real-time sensor status and generating alarm notifications.
- Verified the operation of each module through functional simulation and hardware testing.
- Successfully programmed the RealDigital Boolean Board and demonstrated reliable real-time system operation.

The successful completion of these tasks demonstrates the effectiveness of FPGA technology for implementing intelligent home automation systems.

6.2 Conclusion

This project demonstrates that FPGAs are an ideal platform for high-performance home automation. By replacing sequential microcontroller logic with hardware-level parallel processing, the system achieves near-instantaneous response times for critical safety events. The modular design, coupled with real-time feedback via onboard LEDs, seven-segment displays, and a 3D Blender-based Digital Twin, provides a professional and intuitive solution for modern home safety. The success of this implementation confirms that FPGA technology

provides the speed, scalability, and reliability required for advanced, real-time environmental monitoring.

6.3 Future Scope

Although the implemented system performs real-time monitoring and alert generation, several enhancements can be incorporated in future versions to improve its functionality and expand its application.

The future scope of the project includes:

- Integration of Wi-Fi, Bluetooth, or ZigBee modules for wireless communication.
- Development of a mobile application for remote monitoring and control.
- Integration with IoT cloud platforms for online monitoring and historical data storage.
- Addition of GSM or email notification services for emergency alerts.
- Inclusion of CCTV cameras for visual surveillance and event recording.
- Implementation of fingerprint, RFID, or facial recognition for advanced access control.
- Integration of additional sensors such as smoke, humidity, vibration, and water leakage sensors.
- Development of intelligent energy management features for automatic appliance control.
- Implementation of machine learning algorithms for predictive maintenance and anomaly detection.
- Expansion of the system to support industrial automation and smart building applications.

These improvements would further enhance the intelligence, flexibility, and reliability of the proposed FPGA-based home automation system while extending its applicability to larger and more complex environments.

REFERENCES

- [1] AMD (Xilinx), Spartan-7 FPGA Family Data Sheet (DS189), AMD Inc. [Online]. Available: AMD Documentation
- [2] RealDigital, Boolean Board Complete Reference Manual. [Online]. Available: Boolean Board Reference Manual
- [3] ELECFREAKS, HC-SR04 Ultrasonic Ranging Module Datasheet. [Online]. Available: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>
- [4] Hanwei Electronics Co., Ltd., MQ-2 Gas Sensor Technical Data Sheet. [Online]. Available: <https://www.handsontec.com/dataspecs/MQ-2.pdf>
- [5] Arduino Modules Info, "KY-028 Digital Temperature Sensor Module — Wiring & Code." [Online]. Available: <https://arduinomodules.info/ky-028-digital-temperature-sensor-module/>
- [6] Datasheet Hub, "IR Flame Sensor Module Datasheet." [Online]. Available: <https://www.datasheethub.com/ir-flame-sensor-module/>
- [7] Texas Instruments, LM393 Low-Power Dual Comparator Datasheet. [Online]. Available: <https://www.ti.com/product/LM393>
- [8] Blender Foundation, Blender User Manual. [Online]. Available: <https://docs.blender.org/manual/en/latest>
- [9] Blender Foundation, Blender Python API Documentation. [Online]. Available: Blender Python API Documentation
- [10] M. S. Ahmed, R. Mukherjee, P. Ghosh, S. Nayemuzzaman, and P. Sundaravadivel, "FPGA-based assistive framework for smart home automation," in Proc. 2022 IEEE 15th Dallas Circuit and System Conf. (DCAS), Dallas, TX, USA, 2022, doi: 10.1109/DCAS53974.2022.9845625. [Online]. Available: <https://ieeexplore.ieee.org/document/9845625>
- [11] S. Sharma, J. Boddu, G. S. Charan, S. Sharma, S. Sivanantham, and K. Sivasankaran, "Home automation through FPGA controller," in Proc. 2015 Online Int. Conf. on Green Engineering and Technologies (IC-GET), Coimbatore, India, 2015, doi: 10.1109/IC-GET.2015.7453813. [Online]. Available: <https://ieeexplore.ieee.org/document/7453813>